⑥
⇒ Compute log probabilities of these responses:

log_probs = compute_log_probs(prompts, responses, model)

↳ return I batch Trial

params

⑦
⇒ compute loss so that we can use to update the model

loss = compute_loss(log_probs, deltas, mode="naive")

↳ return loss

⑧
⇒ optional: use explicit KL penalty to regularize the model

KL_penalty = compute_KL_penalty(log_probs, ref_log_probs)

KL(P || q) = E_{x~P}[q(x)/P(x) − log(q(x)/P(x)) − 1], because E_{x~P}[q(x)/P(x)] = 1

summary steps #1 ↔ #8

①: generate response

②: Compute rewards R and δ (centered/normalized/max rewards) // delta

③: Compute log prob of responses

④: Compute loss from log probs and δ (naive, cliped, ~~unclipped~~) (loss) (logs) = logs → unclipped)

sampling trajectory → reward trajectory → prob-weighted reward

→ update policy

old policy model

Def freezing_parameters(): pre-training functions depends on old_para in

⇒ Motivation: in GRPO, you'll see ratios: P(a|s) / P_old(a|s)

⇒ when you're optimizing, it is important to freeze and not differentiate through P_old.

⇒ w = torch.nn.sigmoid(xw) / P_old = torch.nn.sigmoid()(w) /

ratio = P / P_old / ratio.backward() / grad = w.grad / Do it() properly

create old-P as constant with torch.no_grad(): P_old = torch.nn.sigmoid()(w)

Def run_policy_gradient(): refer to: CS224R_Deep Reinforcement Learning

=> train a model using policy gradient

=> return: { path to image of the learning curve
{ path to log file.

①: Define data/model/optimizer

②: Sample response and evaluate their rewards

calculate => responses/rewards/advs

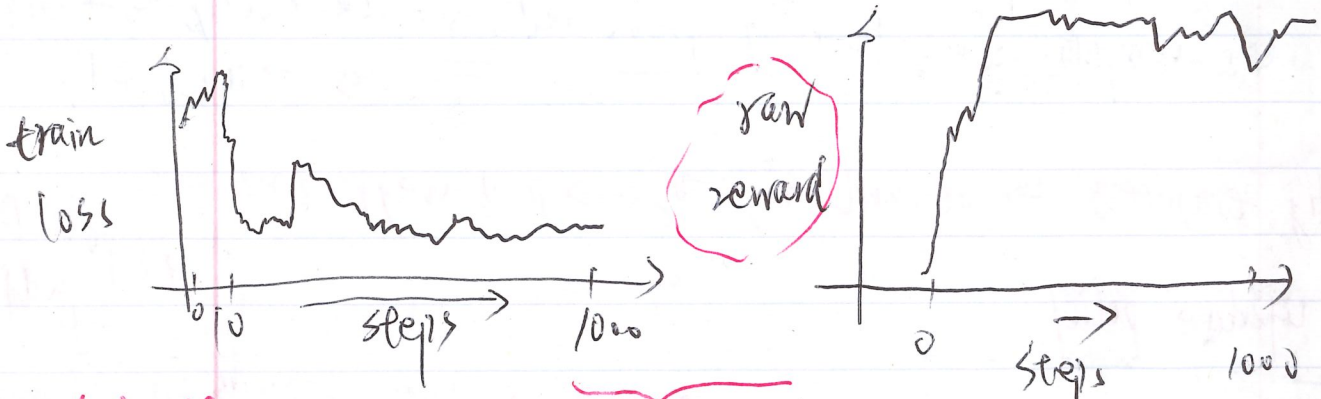
③: compute KL-penalty under reference model/current model

④: take steps given the responses (freeze while do inner steps)

⑤: print training info

⑥: Backprob and update parameters

⑦: plot: loss/reward



not learnt sorting very well

still have local optima issue